

Procesamiento de video 4K con filtro sepia en Python y PyTorch

Institución: UNTDF

Carrera: Lic. en Sistemas, 5to año

Materia: Sistemas Paralelos

Docente: MsC. Federico González Brizzio

Abstract

Este informe compara tres implementaciones de un filtro sepia aplicado sobre un video 4K de 30 segundos: una versión secuencial en Python, una versión PyTorch CPU y una versión PyTorch GPU. La versión secuencial se usa como línea base para calcular el speed-up, siguiendo el criterio de la consigna. El procesamiento se realiza frame por frame para no cargar el video completo en memoria, y el audio se reincorpora al final con ffmpeg.

El video procesado tiene resolución 3840x2160, una duración de 30.013 s, 1799 frames y 59.94 FPS. Los resultados muestran una mejora clara al pasar de Python puro a tensores PyTorch: PyTorch CPU obtiene un speed-up de 17.38x y PyTorch GPU obtiene un speed-up de 33.56x respecto del secuencial en tiempo total de pipeline. Aun así, ninguna versión alcanza tiempo real frente a los 59.94 FPS del video original; el mejor caso, PyTorch GPU, procesa 6.33 FPS efectivos.

1. Introducción

El procesamiento de video puede analizarse como una secuencia de transformaciones sobre imágenes: cada frame se lee, se procesa como imagen estática, se escribe en un nuevo video y luego se recompone la pista de audio. En un video 4K, esta operación tiene un costo importante porque cada frame 3840x2160 con tres canales uint8 ocupa alrededor de 25 MB sin compresión.

La consigna de la sección 10.5 pide procesar un video 4K de aproximadamente 30 segundos con un filtro elegido, implementar una versión secuencial en Python, una versión PyTorch CPU y una versión PyTorch GPU si el hardware lo permite. También exige no cargar el video completo en memoria, reconstruir el video final, reincorporar audio si existe y medir tiempos de lectura, filtrado, escritura y pipeline total.

El filtro elegido fue **sepia**, porque es visible, simple de explicar y puede implementarse de forma equivalente tanto con bucles Python como con tensores PyTorch.

2. Metodología

2.1 Entorno de ejecución

Propiedad	Valor
CPU	Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
Núcleos físicos	4

Propiedad	Valor
Procesadores lógicos	8
Threads por núcleo	2
RAM	15.45 GiB
Sistema operativo	Manjaro Linux, kernel 6.12.91-1-MANJARO
Python	3.12.11
OpenCV	4.13.0.92
PyTorch	2.11.0+cu128
CUDA del wheel PyTorch	12.8
GPU	NVIDIA GeForce GTX 1650, usada en la corrida PyTorch GPU registrada

Durante una verificación posterior a la ejecución, `nvidia-smi` no pudo comunicarse con el driver NVIDIA. Por ese motivo, para reproducir la medición PyTorch GPU es necesario tener el driver operativo. Los resultados conservados incluyen la corrida PyTorch GPU y sus tiempos de transferencia y cómputo.

2.2 Video utilizado

El archivo original fue:

`tp-final/video_facultad.webm`

Características del video original:

Dato	Valor
Resolución	3840x2160
Duración original	00:50:15.94
FPS	59.94
Codec de video	AV1
Codec de audio	Opus

OpenCV detectaba la metadata del WebM original, pero no podía decodificar correctamente sus frames AV1 en este entorno. Para poder procesar frame por frame con OpenCV, se generó un clip de trabajo de 30 segundos en H.264/MP4, manteniendo audio:

`tp-final/entrada/video_facultad_30s_h264.mp4`

Características del clip procesado:

Dato	Valor
Resolución	3840x2160
Duración	30.013 s
Frames procesados	1799
FPS	59.94
Codec de video de entrada	H.264
Codec de audio de entrada	AAC
Tamaño del archivo de entrada	38 MB

La conversión previa no forma parte del tiempo de filtrado. Se usa como preparación de entrada para que las tres implementaciones procesen exactamente el mismo tramo de video.

2.3 Filtro sepia

El filtro sepia transforma cada píxel RGB mediante una combinación lineal de sus canales. Como OpenCV lee y escribe frames en formato BGR, el código toma los canales en orden B, G, R, calcula el resultado sepia y lo vuelve a escribir como B', G', R'.

La fórmula usada en las tres implementaciones fue:

$$\begin{aligned} R' &= \min(255, (393R + 769G + 189B) // 1000) \\ G' &= \min(255, (349R + 686G + 168B) // 1000) \\ B' &= \min(255, (272R + 534G + 131B) // 1000) \end{aligned}$$

Se usó aritmética entera para reducir diferencias de redondeo entre implementaciones.

2.4 Implementaciones comparadas

1. **Secuencial:** recorre byte a byte cada frame con bucles Python. No usa NumPy ni PyTorch para calcular el filtro. OpenCV se usa solamente para leer y escribir video.
2. **PyTorch CPU:** convierte cada frame a tensor y aplica la fórmula sepia mediante operaciones tensoriales en CPU. Se configuraron 4 workers, coincidiendo con los núcleos físicos.
3. **PyTorch GPU:** transfiere el frame actual a CUDA, aplica el filtro con tensores en GPU, sincroniza antes de cerrar los tiempos y devuelve el frame a CPU para escribirlo.

No se implementó una variante NumPy. La aparición de conversiones como `torch.from_numpy(...)` responde a la interfaz entre OpenCV y PyTorch: OpenCV entrega frames como arreglos, y PyTorch los convierte a tensores. Eso no implica una implementación NumPy del filtro.

2.5 Medición

Se hicieron 3 corridas por método y se reporta el promedio. Para cada método se midió:

- tiempo de lectura o decodificación de frames;
- tiempo de filtrado;
- tiempo de escritura o codificación;
- tiempo total del pipeline;
- FPS efectivos;
- memoria pico aproximada;
- checksum y hash de salida;
- tiempo de reincorporación de audio, medido aparte.

En PyTorch GPU también se separó:

- transferencia CPU -> GPU;
- cómputo en GPU;
- transferencia GPU -> CPU.

El speed-up se calculó siempre contra la versión secuencial:

$\text{speed-up} = \text{tiempo_total_secuencial} / \text{tiempo_total_metodo}$

El tiempo total de pipeline incluye lectura, filtrado y escritura. El merge de audio se mide aparte y no se suma al pipeline.

2.6 Manejo de memoria

El video no se carga completo en memoria. La estrategia fue:

leer frame -> aplicar filtro -> escribir frame -> liberar referencias -> continuar

En GPU se transfirió solamente el frame actual. No se conservaron todos los frames ni todos los tensores intermedios en memoria del dispositivo.

3. Resultados

3.1 Tabla principal

Método	Frames	Resolución	FPS original	Lectura (s)	Filtrado (s)	Escritura (s)	Pipeline total (s)	Minutos	FPS efectivos	Speed-up	Memoria pico (MB)
secuencial	1799	3840x2160	59.94	10.709	9528.508	74.851	9614.069	160.23	0.187	1.000	592.98

Método	Frame s	Resolución	FPS original	Lectura (s)	Filtrado (s)	Escritur a (s)	Pipeline total (s)	Minut os	FPS efectivo s	Speed-u p	Memori a pico (MB)
PyTorch CPU	1799	3840x2160	59.94	13.422	447.949	91.758	553.128	9.22	3.268	17.381	1378.95
PyTorch GPU	1799	3840x2160	59.94	19.371	171.182	95.957	286.509	4.78	6.331	33.556	1430.34

3.2 Distribución del tiempo por etapa

Método	Lectura (%)	Filtrado (%)	Escritura (%)
secuencial	0.11	99.11	0.78
PyTorch CPU	2.43	80.98	16.59
PyTorch GPU	6.76	59.75	33.49

En el secuencial, casi todo el tiempo se consume en el filtro. En PyTorch CPU y GPU, al acelerarse el filtrado, la escritura del video empieza a representar una parte más visible del pipeline.

3.3 Detalle de PyTorch GPU

Etapas GPU	Tiempo (s)	Porcentaje del filtrado GPU
Transferencia CPU -> GPU	71.806	41.95
Cómputo GPU	67.816	39.62
Transferencia GPU -> CPU	31.560	18.44
Transferencia total	103.366	60.38
Filtrado GPU total	171.182	100.00

El cómputo puro en GPU fue menor que el tiempo total de transferencia. Esto muestra que, para este pipeline, usar GPU no elimina el costo de mover cada frame entre CPU y GPU. Aun así, PyTorch GPU fue el método más rápido en tiempo total.

3.4 Comparación CPU vs GPU

Comparación	Relación
PyTorch CPU vs secuencial, pipeline completo	17.381x
PyTorch GPU vs secuencial, pipeline completo	33.556x
PyTorch GPU vs PyTorch CPU, pipeline completo	1.931x

Comparación	Relación
PyTorch CPU vs secuencial, solo filtrado	21.271x
PyTorch GPU vs secuencial, solo filtrado	55.663x
PyTorch GPU vs PyTorch CPU, solo filtrado	2.617x
Cómputo GPU vs filtrado PyTorch CPU	6.605x

La GPU mejora claramente respecto de PyTorch CPU, pero la diferencia en pipeline completo es menor que la diferencia de cómputo puro por el costo de transferencias y escritura.

3.5 FPS efectivos y tiempo real

Método	FPS efectivos	Porcentaje del FPS original
secuencial	0.187	0.31
PyTorch CPU	3.268	5.45
PyTorch GPU	6.331	10.56

El video original reproduce a 59.94 FPS. Ninguna implementación llega a tiempo real. PyTorch GPU fue la más rápida, pero alcanza solo el 10.56% del FPS original.

3.6 Equivalencia de salidas

Método	Checksum	Hash
secuencial	3788871516465	7685a7d3304f5e10
PyTorch CPU	3788871516465	7685a7d3304f5e10
PyTorch GPU	3788871516465	7685a7d3304f5e10

Los tres métodos produjeron el mismo checksum y el mismo hash de salida. Esto indica que aplicaron el mismo filtro sobre los mismos frames y que la comparación de tiempos no está mezclada con diferencias de resultado.

3.7 Reconstrucción del video y audio

Por cada método se generó un video sin audio y luego una versión con audio:

```
resultados/videos/sepia_secuencial_con_audio.mp4
resultados/videos/sepia_pytorch_cpu_con_audio.mp4
resultados/videos/sepia_pytorch_gpu_con_audio.mp4
```

El audio se reincorporó con ffmpeg usando la pista del clip de entrada entrada/video_facultad_30s_h264.mp4. Ese paso se registró aparte:

Método	Merge de audio (s)
secuencial	5.530
PyTorch CPU	3.412
PyTorch GPU	4.174

El video final con audio de PyTorch GPU mantiene duración 30.01 s, resolución 3840x2160, 59.94 FPS y audio AAC estéreo a 48000 Hz.

3.8 Evidencia visual

La evidencia visual del filtro no se incluye como imágenes estáticas dentro del informe, porque la entrega contiene los videos procesados. Las salidas con audio quedan en:

```
resultados/videos/sepia_secuencial_con_audio.mp4
resultados/videos/sepia_pytorch_cpu_con_audio.mp4
resultados/videos/sepia_pytorch_gpu_con_audio.mp4
```

Esos archivos permiten verificar directamente el efecto sepia sobre el video completo y no solo sobre un frame aislado.

4. Análisis

4.1 Línea base secuencial

La versión secuencial cumple el rol de línea base. Su tiempo total promedio fue 9614.069 s, alrededor de 160.23 minutos para procesar 30 segundos de video. El dato principal es que el 99.11% del pipeline corresponde al filtrado. Esto confirma que el cuello de botella no está en leer o escribir video, sino en recorrer píxel por píxel desde Python.

Este resultado es esperable: cada frame 4K tiene más de 8 millones de píxeles, y el clip contiene 1799 frames. Aunque la fórmula sepia es simple, la cantidad de iteraciones vuelve inviable al secuencial para procesar video 4K en tiempos prácticos.

4.2 PyTorch CPU

PyTorch CPU reduce el pipeline a 553.128 s, con un speed-up de 17.381x respecto del secuencial. El filtrado baja de 9528.508 s a 447.949 s, lo que equivale a una mejora de 21.271x en la etapa de cómputo del filtro.

La mejora se explica porque PyTorch aplica operaciones tensoriales sobre el frame completo y evita el costo de los bucles Python por píxel. Se usaron 4 workers,

correspondientes a los núcleos físicos de la CPU. No se usaron 8 workers porque esos corresponden a hilos lógicos y mezclarían el análisis con SMT/concurrencia.

A medida que el filtrado se acelera, la escritura del video pasa a pesar más: en PyTorch CPU representa 16.59% del pipeline. En el secuencial ese costo quedaba oculto por el tiempo del bucle Python.

4.3 PyTorch GPU

PyTorch GPU fue el mejor método en tiempo total: 286.509 s, con un speed-up de 33.556x respecto del secuencial y una mejora de 1.931x frente a PyTorch CPU en pipeline completo.

La mejora es mayor cuando se mira solo el filtrado. PyTorch GPU reduce el filtrado total a 171.182 s, y el cómputo puro en GPU toma 67.816 s. Comparado con el filtrado de PyTorch CPU, el cómputo GPU es 6.605x más rápido.

Sin embargo, el costo de transferencia es muy importante. La transferencia CPU->GPU y GPU->CPU suma 103.366 s, es decir el 60.38% del tiempo de filtrado GPU. En otras palabras, la GPU acelera el cálculo, pero el pipeline sigue condicionado por mover frames de 4K entre memoria de CPU y memoria de GPU, y por devolverlos a CPU para escribir el video con OpenCV.

4.4 Límite de tiempo real

Aunque PyTorch GPU fue el método más rápido, no alcanza tiempo real. El video original reproduce a 59.94 FPS, mientras que PyTorch GPU procesa 6.331 FPS efectivos. Esto equivale al 10.56% del FPS original.

El resultado muestra una diferencia importante entre acelerar el filtro y acelerar todo el pipeline. Para acercarse a tiempo real habría que reducir transferencias, optimizar escritura/codificación o usar una estrategia donde la decodificación, procesamiento y codificación permanezcan más cerca de GPU.

4.5 Manejo de memoria

La estrategia frame por frame permitió procesar el clip sin cargar los 1799 frames en memoria. Si se cargara el video completo en uint8, solo los frames sin comprimir requerirían decenas de GB. Si además se guardaran tensores intermedios, el uso crecería todavía más.

La memoria pico reportada fue:

Método	Memoria pico (MB)
secuencial	592.98
PyTorch CPU	1378.95
PyTorch GPU	1430.34

Las versiones PyTorch usan más memoria porque crean tensores intermedios y, en GPU, además requieren buffers de transferencia.

4.6 Sobre la conversión previa del video

La conversión del WebM AV1 original a MP4/H.264 no se incluyó en los tiempos de pipeline. Fue una preparación necesaria porque OpenCV no pudo leer frames del archivo AV1 en este entorno. El clip convertido mantiene la resolución 4K, los 59.94 FPS, el tramo de 30 segundos y una pista de audio derivada del original.

Esto no cambia la comparación entre métodos, porque los tres procesan exactamente el mismo archivo de entrada convertido.

5. Conclusiones

El filtro sepia se implementó de forma equivalente en las tres variantes. La coincidencia de checksum y hash confirma que las salidas son iguales entre secuencial, PyTorch CPU y PyTorch GPU.

La versión secuencial es adecuada como línea base conceptual, pero no como estrategia práctica para video 4K: tarda más de dos horas y media por corrida de 30 segundos. PyTorch CPU reduce el tiempo a alrededor de nueve minutos, mostrando el impacto de expresar el cálculo como operaciones tensoriales.

PyTorch GPU fue la estrategia más rápida. Logró un speed-up de 33.556x respecto del secuencial y casi duplicó el rendimiento de PyTorch CPU en pipeline completo. Aun así, las transferencias CPU-GPU y la escritura del video limitan la mejora observada. El cómputo GPU puro es rápido, pero el pipeline completo todavía queda lejos de tiempo real.

Para este caso, la estrategia más adecuada es PyTorch GPU si el driver CUDA está operativo. Si la GPU no está disponible, PyTorch CPU es la mejor alternativa y mantiene una mejora muy grande frente al secuencial.